

TECHNICAL SUMMARY SHEET

Software application logger library that is easy to use, robust, fail-safe and reliable.

Technical Advantage:

The primary benefit using this fail-safe C-Program API library over other application loggers, is that it does extensive runtime validation to prevent the application from crashing due to common programmers printf() style pitfalls or unpredictable execution segmentation faults. (e.g., string type %s using NULL, 0 or negative value).

During runtime, the API decode, analyze and apply 18 different Bound Parameter verification techniques (see *WandiSSAL-Bound-Capabilities.pdf*) to ensure log messages are used accurately, fail-safe and reliable. If a runtime validation fails the API generates an error code and simple returns, allowing the application to continue normal processing (no need for the application to check/stop processing).

In addition, the library was designed and implemented using secure coding practices and techniques. It was further tested with Common Weakness Enumeration (CWE) security scanning tools to identify CWE vulnerabilities and weaknesses applicable to the library codebase. Based on those finding the codebase was enhanced to incorporate CWE recommended solutions to reduce security risks and increase software reliability. This includes but not limited to the following CWEs:

- CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition').
- CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow').
- CWE-134: Use of Externally-Controlled Format String.
- CWE-126: Buffer Over-read.
- CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer.
- CWE-190: Integer Overflow or Wraparound.
- CWE-787: Out-of-bounds Write.
- CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
- CWE-20: Improper Input Validation.
- CWE-284: Improper Access Control.
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.
- CWE-770: Allocation of Resources Without Limits or Throttling.

API Usage:

Provides a simple printf() style function interface commonly used to generate application log messages:

```
WandiSSAL_LoggerAppsMsg( DevProd, LogicalName, SeverityLevel, "Format string specification, string specifier value %s", "string value" );
```

1. DevProd controls generating development and/or production log messages.
2. LogicalName indicates file name, function name or common name across multiple files.
3. SeverityLevel indicates different levels of severity importance.
4. The remainder of the arguments are similar to printf() style specification.
5. Uses runtime Bound Parameter validations to ensure fail-safe application execution:
 - a. Use only LogicalName defined for use by the API.
 - b. Use only SeverityLevel defined for use by the API.
 - c. Decode printf() format string specification for string specifiers and match to corresponding log message arguments.
 - d. Validate length of format string specification to prevent use of unbounded strings.
 - e. Validate length of string argument to prevent the use of improper NULL terminated strings.

- f. Validate specific ASCII characters to avoid use of non-printable characters in log message.
- g. Validate against an acceptable set of format string specifiers to prevent unsafe and malicious use of attacker string specifiers.
- h. Validate LogicalName and SeverityLevel for supported values.
- i. Prevent use of invalid NULL, 0, negative values for %s format string specifier.
- j. Block the use of untrusted or malicious format string specifiers.
- k. Block the use of ill-formed or non-standard format specifiers.
- l. When API fails validation, it generates a status code and safely returns.
- m. Application does not need to check API return status.
- n. Enable or disable log message processing and generation.

Bound Parameter validation details are provided in *WandiSSAL-Bound-Capabilities.pdf* and programming examples are listed the *WandiSSAL-bound-example.c* program.

6. Standard log file name includes: [pid_date_timestamp_sequence-number.log]:

3550_09012026_17:14:20_001.log

7. Standard output log file content format:

02/09/2026 12:01:55 [3650:sample_program] [DATABASE] [INFO] Message that generates 1 string and 2 integer arguments [String Argument], [98] and [99]

where:

"02/09/2026 12:01:55" date and timestamp when the log message was generated.
 "[3650:sample_program]" application process id and name.
 "[DATABASE]" logical message group for related application log messages.
 "[INFO]" severity for log message.

The remainder of the output is derived from the C-Programming Language printf() format string specification and corresponding arguments.

Contact: info@istech.com

Documentations and Download: <https://github.com/wandi-ssal/fail-safe-application-logger>